ChatGPT

# Space Netra Mini-Project Planning Pack

## plan.md

**Scope:** The first goal is a working demo on a small area in Chhatrapati Sambhajinagar. We should **preserve the existing code and vector DB**. Start by clearly defining an AOI (e.g. a few km² around a known location in the city) and verifying that open satellite imagery exists for it. Sentinel-2 (10 m) data is freely available with ~5-day revisit【72†L77-L80】【50†L159-L163】, and Landsat (30 m) is also free【72†L77-L80】. Use these first; higher-resolution commercial imagery (PlanetScope ~3 m, Maxar ~0.5 m) is paid. Note that at 10 m resolution only very large structures are visible (only ~0.2% of buildings cover a 4×4 pixel area【70†L117-L124】), so we must adjust expectations about what "change" means in this demo.

**Week-Long Schedule (target):**

- **Day 1:** Back up the current IDE project (including `vector_db/`). Inspect files (`app.py`, ingestion scripts, etc.). Define AOI boundary (e.g. using Folium or GeoJSON) around part of Sambhajinagar. Search for recent Sentinel-2 scenes over that AOI via Microsoft Planetary Computer STAC or Copernicus Open Hub. Check cloud cover.
- **Day 2:** Download at least two cloud-free images (a "before" and "after" date, e.g. one year apart). Store raw data (as TIFF or NumPy arrays).
- **Day 3:** Preprocess images: mask clouds/shadows (e.g. simple threshold or band QA), and co-register them. Use **rasterio** to reproject/warp images to a common CRS and grid (misalignment even <1 px causes big errors【64†L21-L24】【64†L45-L48】). Save aligned, masked images.
- **Day 4:** Compute change metrics. For a baseline, use pixel-difference or structural similarity (SSIM) on grayscale or NDVI. Generate a binary **change mask** by thresholding.
- **Day 5:** Integrate into Streamlit app: add UI to draw/select AOI, pick dates, and display the change mask overlay on a map. Also show stats (e.g. change area in hectares). (At 10 m GSD each pixel ~100 m² = 0.01 ha.)
- **Day 6:** Review results in a few sample areas; validate qualitatively (e.g. with Google satellite basemap). Tweak thresholds or band combinations to reduce noise.
- **Day 7:** Finalize demo and documentation. Prepare a brief report. Ensure the code runs from a clean state given parameters. Confirm deliverables: code, `vector_db/`, and pipeline scripts.

The emphasis is on a working proof-of-concept, not an extensive feature list. The main dependency is finding suitable image pairs on Day 1; do not invest in analysis without verifying data availability. (Sentinel-2 data is indeed free and open for any use【17†L32-L40】【72†L77-L80】, but check local presence and cloud cover.)

**Success criteria:** The app should show "detected change" areas on the map for our chosen AOI/dates, and output a list of changed regions or metrics. Even simple outputs (like a heatmap or pixel count) are acceptable if clearly explained. Avoid claims about identifying specific building changes at 10 m resolution unless we have very coarse definition of "change" (e.g. urban expansion area).

# architecture.md

We propose a modular architecture separating **image search** from **change detection**:

- **Streamlit UI:** One web app front-end (e.g. `app.py`) with controls to choose AOI and feature (search vs change-detection), and to display results on a map/plots.
- **Image Search Pipeline:** Uses the existing vector DB and CLIP/FAISS stack. It handles user text queries (e.g. “river”, “road”) by embedding with CLIP and retrieving similar indexed satellite images. CLIP (Contrastive Language-Image Pretraining) encodes images and text into the same vector space 【39†L10-L18】. FAISS indexes these vectors for nearest-neighbor search【43†L154-L161】. This pipeline will remain unchanged.
- **Change-Detection Pipeline:** A new separate workflow. Responsibilities:
- **AOI & Data:** Take a user-defined area and find/download two satellite scenes (before and after).
- **Preprocessing:** Mask clouds, reproject/align the images to a common grid (treat one image as reference)【64†L21-L24】【64†L45-L48】.
- **Analysis:** Compute differences (e.g. SSIM/NDVI) and threshold to create a change mask.
- **Postprocess:** Optionally vectorize changes or filter noise.
- **Output:** Provide results back to the UI (change mask overlay, area statistics, cropped before/after views, etc.).

Each pipeline should have its own folder (e.g. `image_search/` vs `change_detect/`) and scripts. The existing `vector_db/` is **not modified**; back it up. New code for change detection should read the AOI parameter (e.g. geojson or bounding box) and pass data to the Streamlit app via function calls or saved files.

**Data Flow Summary:** User → Streamlit → [SEARCH: vector_db + CLIP/FAISS → results] and [CHANGE: AOI + STAC API → raw images → preprocessing → change mask → results].

# tech.md

**Programming Language:** Python 3.x (already used).

**Key Libraries and Tools:**

- **Streamlit**: Web dashboard (already in project requirements). Allows rapid GUI for sliders, maps, charts.
- **Folium / Streamlit-Folium:** For interactive Leaflet maps within Streamlit. Users can draw AOIs or pan the map.
- **GeoPy:** Geocoding services if needed (e.g. convert place names to coordinates).
- **rasterio**: Geospatial raster I/O and reprojection (for aligning images to a reference grid)【64†L21-L24】.
- **NumPy, Pillow (PIL), scikit-image, OpenCV:** Image processing and array handling. scikit-image includes SSIM and filters.
- **PyTorch + OpenAI CLIP:** (Already present) CLIP provides pretrained encoders. We use it to embed images/text【39†L10-L18】.
- **FAISS (faiss-cpu):** Fast vector similarity search index【43†L154-L161】 for our CLIP embeddings.

- **Requests / Earth Engine API:** For downloading images (if using GEE or other API). Alternatively, use `planetary-computer` Python client or `sentinelsat`.

**Supported Data Formats:** GeoTIFF or JPEG2000 for satellite, GeoJSON for AOI, NumPy arrays for processing. Use CRS EPSG:4326 or UTM as needed.

**System Design Notes:** The app can run locally. If using the Microsoft Planetary Computer STAC, obtain an API key (free) and query their catalog. Alternatively use Copernicus Data Space Hub (no key needed, but REST calls). These data sources guarantee free access under Copernicus terms【17†L32-L40】. Do **not** attempt to scrape or bulk download from sources like Bhuvan, as those terms forbid it【13†L11-L19】【13†L23-L26】.

**CLIP & FAISS Justification:** CLIP's joint image-text space lets the app match semantic queries to satellite images【39†L10-L18】. FAISS (Facebook AI Similarity Search) is a library optimized for fast nearest-neighbor search in large vector datasets【43†L154-L161】. Both have active Python support and match the original project's setup.

# pipeline.md

**Step 1: Define AOI and Dates.** The user selects (or draws) a polygon around the target area (Chhatrapati Sambhajinagar small region). Select two dates (past and recent) where clear imagery is expected.

**Step 2: Acquire Satellite Scenes.** Query a satellite catalog (e.g. MS Planetary Computer STAC or Copernicus Open Access Hub) for Sentinel-2 (or Landsat) scenes intersecting the AOI. Filter out cloudy scenes using metadata. Download the selected images (e.g. Sentinel-2 Level-2A) as GeoTIFFs.

**Step 3: Preprocessing & Alignment.** - **Cloud Masking:** If clouds appear, mask them out (simple threshold on quality band or NIR difference).<br> - **Co-registration:** Use *rasterio* to warp the second image onto the first image's spatial grid【64†L21-L24】【64†L45-L48】. This means they share the same CRS, origin, resolution, and extent. Alignment is crucial: even a one-pixel offset can mislabel unchanged areas as changed【64†L21-L24】. Choose one image's transform as the reference "grid spec" and reproject the other to match it exactly.

**Step 4: Change Detection Algorithm.** Convert aligned images to arrays (optionally compute an index like NDVI). Compute a per-pixel difference measure. Options include:

- **Absolute Difference / Threshold:** For grayscale or NDVI images, subtract and threshold.
- **Structural Similarity (SSIM):** Compute SSIM map (scikit-image) to highlight structural changes. Low SSIM indicates change.
- **Simple Band Ratio:** Compare band combinations (e.g. new-buildings often change surface reflectance).

Create a binary mask of "change" pixels by thresholding the difference or SSIM map. Optionally apply morphological filters to remove noise. Calculate change area: each 10 m pixel $\approx 100\ m^2$ (0.01 ha), so total changed pixels ×0.01 = area in hectares.

**Step 5: Visualization.** Overlay the change mask on a map (Folium). For example, mark changed areas in semi-transparent red. Show before/after thumbnails of a few change hotspots. Display metrics (e.g. changed area). Include legends and allow toggling layers.

*Note:* This pipeline runs separately from the CLIP search. It may be triggered by a different menu option in the Streamlit app. The intermediate steps (alignment, mask creation) should be written as separate functions for clarity.

## data_sources.md

We will rely on **open-access satellite imagery**:

- **ESA Copernicus Sentinel-2:** 13-band multispectral data (visible+NIR at 10 m, 20 m, 60 m)【50†L146-L154】. Copernicus policy grants *free, full, and open* access to all Sentinel data【17†L32-L40】. Use the 10 m bands (B02, B03, B04, B08) for high-resolution detail.
- **USGS Landsat 8/9:** 30 m multispectral (15 m panchromatic) data, free since 2008【20†L743-L746】. Useful for older imagery if needed.
- **Microsoft Planetary Computer STAC:** Provides a catalog and API for Sentinel-2, Landsat, and other datasets. No cost for data, but subject to Copernicus license.
- **Copernicus Data Space Ecosystem:** Official EU portal for Sentinel data (no API key needed). Copernicus terms allow any use【17†L32-L40】.
- **Planet Scope (Planet Labs) / SpaceNet: Only if** higher resolution is absolutely needed. PlanetScope (~3 m) data is paid, but SpaceNet7 has released a 4 m, multi-temporal building-change dataset 【33†L28-L32】. If training, we could sample from SpaceNet7's open dataset (24 monthly images at 4 m over 100 cities).
- **Bhuvan/Bhoonidhi (NRSC/ISRO):** Indian portals provide high-res views, but their EULAs strictly forbid copying or bulk downloads【13†L11-L19】【13†L23-L26】. These can be used only for visual reference.

In practice, we will **start with Sentinel-2** via Planetary Computer or Copernicus Hub (free, 10 m, ~5-day revisit【50†L159-L163】【72†L77-L80】). If needed later for a pilot, one could explore buying Planet or Maxar images.

Other data: If labeling or validation is needed, consider **OpenStreetMap** footprints or municipal datasets for Chh.Sambhajinagar as ground truth, but note these are vectors, not raw imagery. They could help interpret detected changes (e.g. new buildings vs misfires).

## model_strategy.md

Given the one-week mini-project scope, **we will not train a new deep model initially**. Instead, use classic image-difference methods for explainability. However, we outline options if we had more time:

- **No-Training Baseline:** Directly compare pixels or features (SSIM) between dates. This avoids needing labeled data.
- **Pretrained Model / Fine-tuning:** If we decide to train a change detector, possible datasets include:

- **OSCD (Onera Satellite Change Detection):** 24 Sentinel-2 image pairs with pixel-level urban-change ground truth【25†L33-L39】. It focuses on detecting new buildings/roads. However, it is small, so performance will be limited.
- **SpaceNet 7 (Multi-Temporal Urban Development):** 4 m Planet images with building labels 【33†L28-L32】. Contains 24 monthly images per site. Training on this could learn building-change cues, but note the resolution difference (4 m vs 10 m).
- **Other:** "Bühler datasets" or **EuroSAT** (land use classification) are unrelated.

- **GAN/SR Models:** Super-resolution methods (e.g. SEN2SR) claim to upsample Sentinel-2【70†L200-L205】, but they only guess detail and cannot truly recover sub-pixel structures【70†L200-L205】. Not recommended for detection.

- **Metrics/Evaluation:** Without ground truth for our AOI, evaluate changes manually or by cross-checking with very-high-res basemap (e.g. Google). If we used a dataset like OSCD, metrics like F1-score or IoU (as in its leaderboard) could be used. For SpaceNet-style tasks, the SCOT metric (space-time IoU) is used【33†L28-L32】.

**Key Trade-offs:** At 10 m resolution, small buildings rarely cover even a 2×2 pixel area【70†L117-L124】. Thus if "change" means single-building construction, no method (even trained) will reliably detect them. Instead, focus on **coarser changes** (new cleared areas, expanded blocks). If we had unlimited time, we would still treat training as a future enhancement after validating the pipeline on few examples.

## open_questions.md

Some important unresolved issues that should be clarified **before implementation**:

- **Change Definition:** *What exactly counts as a "change"?* Are we targeting urban expansion (e.g. large new roads or cleared land) or individual buildings? Sentinel-2 (10 m) cannot resolve small buildings – expert analysis suggests 50 cm resolution is needed for reliable building outlines【70†L145-L153】. If evaluators expect to see new houses, we should note this limitation.
- **AOI and Extent:** *Which specific boundaries?* Even within Chh.Sambhajinagar, we need precise coordinates. How large an area ($km^2$) is feasible to download and process? A too-large area will be slow.
- **Dates and Seasonality:** *Which dates to compare?* Ideally, choose the same season to reduce vegetation differences. We must find two cloud-free dates; availability depends on archives.
- **Image Preprocessing:** *Clouds and alignment.* We should ask how to handle clouds. Possibly skip cloudy pixels or choose scenes with minimal cloud. Also, ensure both images are in the same CRS and aligned – see recommendations【64†L21-L24】【64†L45-L48】.
- **Accuracy & Validation:** *What's a "correct" output?* We lack ground truth for Sambhajinagar. Will the graders trust a visual inspection of results? We may need to present some known-check areas (e.g. if we know a construction happened).
- **Output Format:** *How to present results?* Do we need precise area numbers, or just a heatmap overlay? Likely both: e.g. "Changed area = 0.35 ha" along with map markers.
- **Future Scaling:** *If this goes beyond prototype, what next?* For example, if pushing to "pilot project", we might consider automating multiple AOIs, adding a backend database for change masks, etc. But for now, that is speculative.

These questions will shape design choices. For instance, the resolution issue suggests we set expectations: the demo will highlight broad changes, not detailed building outlines【70†L117-L124】【33†L28-L32】. We should document such assumptions.

# Sources

- Copernicus Sentinel license policy【17†L32-L40】 (free/open data).
- NASA Landsat free/open data policy【20†L743-L746】.
- ISRO Bhuvan Terms (restricts bulk imagery use)【13†L11-L19】【13†L23-L26】.
- CLIP model description【39†L10-L18】, FAISS overview【43†L154-L161】.
- Onera Satellite Change Detection (OSCD) dataset description【25†L33-L39】.
- SpaceNet 7 Challenge summary【33†L28-L32】 (Planet imagery, building tracking).
- Sentinel-2 specifications (10 m bands)【50†L146-L154】 and revisit time【50†L159-L163】.
- Analysis of resolution needed for building detection【70†L117-L124】【70†L148-L153】, Satellite imagery cost vs resolution【72†L77-L80】.
- Geospatial raster alignment best practices【64†L21-L24】【64†L45-L48】.

---